Object-Oriented Programming — Interview Questions

The OOPs questions asked in fresher technical interviews, with short answers to say aloud. Examples use Java; the concepts are the same in C++ and Python (differences are noted).

Answer like this in the interview

Define in one line → **explain** with one real-world example → add **one detail** (a difference, a use case or a limitation). Stop at 30–45 seconds and let the interviewer ask the follow-up.

Questions marked ★ **MOST ASKED** come up in almost every fresher technical round — learn those first.

Top questions to learn first

Four pillars · Class vs object · Overloading vs overriding · Abstract class vs interface · Encapsulation vs abstraction · Types of inheritance · Why no multiple inheritance in Java · Constructor & its types · Runtime polymorphism · Shallow vs deep copy

1 · Basics

1 What is OOP?

A programming approach that models a program as **objects** — each object bundles **data (fields)** and **behaviour (methods)**. It makes code reusable, modular and easier to maintain. Java, C++, Python and C# support OOP.

2 What is a class and what is an object? ★ MOST ASKED

A **class** is a blueprint that defines fields and methods. An **object** is an instance of that class created in memory.

Example: `Car` is a class; `myCar = new Car()` is an object with its own colour and speed. A class takes no memory for data until objects are created.

3 What are the four pillars of OOP? ★ MOST ASKED

- **Encapsulation** — wrap data and methods together and hide the data (private fields, public getters/setters).
- **Abstraction** — show only what an object does, hide how (abstract classes, interfaces).
- **Inheritance** — a child class reuses and extends a parent class (`extends`).
- **Polymorphism** — one name, many forms (method overloading and overriding).

Give one real example each — e.g. an ATM: you use withdraw() (abstraction), the balance is private (encapsulation), SavingsAccount extends Account (inheritance), and withdraw() behaves differently per account type (polymorphism).

Procedural vs object-oriented programming

Point	Procedural (C)	Object-oriented (Java, C++)
Focus	Functions and steps	Objects that hold data + behaviour
Data security	Data is mostly global / open	Data hidden with access modifiers
Reuse	Copy or call functions	Inheritance and composition
Best for	Small, linear programs	Large applications that change over time

2 · Encapsulation & abstraction

4 **What is encapsulation?** ★ MOST ASKED

Binding data and the methods that work on it into one unit (a class) and **restricting direct access** to the data. Fields are `private`; access goes through public **getters and setters** that can validate values.

Benefit: the object protects its own state — e.g. a negative deposit can be rejected.

```
class Account {
    private double balance;            // hidden data

    public void deposit(double amt) {  // controlled access
        if (amt <= 0) throw new IllegalArgumentException("Invalid amount");
        balance += amt;
    }
    public double getBalance() { return balance; }
}
```

5 **What is abstraction?** ★ MOST ASKED

Showing only the **essential features** and hiding the implementation details. A driver uses the steering and brake without knowing how the engine works. In Java it is achieved with **abstract classes** (partial) and **interfaces**.

6 **Encapsulation vs abstraction?** ★ MOST ASKED

Abstraction hides **complexity** — it is about what an object does (design level). **Encapsulation** hides **data** — it is about how the data is protected (implementation level).

One-line version: abstraction = hide implementation; encapsulation = hide data and control access.

Access modifiers in Java

Modifier	Same class	Same package	Subclass (other package)	Everywhere
private	Yes	No	No	No
default (none)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Python note

Python has no enforced private keyword. A single underscore `_x` means "internal by convention"; a double underscore `__x` triggers name mangling (`_ClassName__x`).

3 · Inheritance

7 **What is inheritance?** ★ MOST ASKED

A mechanism where a **child (sub) class acquires the fields and methods of a parent (super) class** and can add or change behaviour. It represents an **IS-A** relationship — `Dog extends Animal`, a Dog IS-A Animal. Benefit: code reuse and a natural hierarchy.

8 **What are the types of inheritance?** ★ MOST ASKED

- **Single** — B extends A.
- **Multilevel** — C extends B, B extends A.
- **Hierarchical** — B and C both extend A.
- **Multiple** — C inherits from A and B (allowed in C++ and Python; in Java only through interfaces).
- **Hybrid** — a mix of the above.

9 **Why doesn't Java support multiple inheritance with classes?** ★ MOST ASKED

To avoid the **diamond problem**: if B and C both override a method of A and D extends both, the compiler cannot decide which version D gets. Java allows a class to implement **multiple interfaces** instead; if two interfaces have the same default method, the class must override it and choose.

10 **What is the `super` keyword?**

It refers to the **immediate parent class**: `super.method()` calls the parent's version of an overridden method, `super.field` accesses a parent field, and `super(args)` calls the parent constructor — it must be the **first statement** in the child constructor.

11 Inheritance (IS-A) vs composition (HAS-A)?

Inheritance: `Car extends Vehicle` — Car IS-A Vehicle. Composition: a Car HAS-A Engine (Engine is a field). Prefer **composition** when the relationship is "has a" — it gives looser coupling and lets you swap parts without changing the class hierarchy.

---

4 · Polymorphism

12 What is polymorphism and what are its types? ★ MOST ASKED

The ability of one method name or object reference to take **many forms**.

- **Compile-time (static)** — method **overloading**; the compiler picks the method from the arguments.
- **Run-time (dynamic)** — method **overriding**; the JVM picks the method from the actual object type at run time.

Method overloading vs method overriding ★ MOST ASKED

Point	Overloading	Overriding
Where	Same class (or inherited)	Parent and child class
Method signature	Same name, **different parameters**	Same name **and** same parameters
Return type	Can differ (parameters must differ)	Same, or a subtype (covariant)
Binding	Compile time (static)	Run time (dynamic)
static / private / final	Can be overloaded	Cannot be overridden
Example	add(int, int) and add(double, double)	Animal.sound() redefined in Dog

13 Explain runtime polymorphism with an example. ★ MOST ASKED

A **parent reference holds a child object**; the overridden method that runs is decided by the **object**, not the reference type. This is called **dynamic method dispatch**.

```
class Animal { void sound() { System.out.println("Some sound"); } }
class Dog extends Animal { @Override void sound() { System.out.println("Bark"); } }

Animal a = new Dog();   // parent reference, child object
a.sound();              // prints "Bark" — decided at run time
```

---

14 Can we overload a method by changing only the return type?

No. `int area()` and `double area()` give a compile error because the call `area()` is ambiguous — the parameter list must differ.

---

15 Can static, private or final methods be overridden?

- **static** — no; redefining it in the child **hides** the parent method (method hiding), it is not overriding.
- **private** — no; it is not visible to the child.
- **final** — no; `final` exists to prevent overriding. (A final class cannot be extended at all.)

---

5 · Abstract class & interface

16 What is an abstract class?

A class declared `abstract` that **cannot be instantiated**. It can have abstract methods (no body) **and** concrete methods, fields and constructors. A subclass must implement all abstract methods or itself be abstract. Use it when related classes share common code.

---

17 What is an interface?

A **contract** that lists what a class must do. Methods are `public abstract` by default; since Java 8 interfaces can have `default` and `static` methods, and since Java 9 private methods. Fields are always `public static final` (constants). A class can implement **many** interfaces.

Abstract class vs interface ★ MOST ASKED

Point	Abstract class	Interface
Methods	Abstract and concrete	Abstract; default/static allowed (Java 8+)
Variables	Any type, any access	Only public static final constants
Constructor	Yes	No
Inheritance	A class extends only one	A class implements many
Keyword	extends	implements
Use when	Classes share code and state (IS-A)	Unrelated classes share a capability (CAN-DO), e.g. Comparable

18 Can an abstract class have a constructor? Can we create its object?

It **can** have a constructor — it runs when a subclass object is created (through `super()`), to initialise common fields. You **cannot** create an object directly with `new AbstractClass()`.

---

6 · Constructors & objects

19 What is a constructor? What are its types? ★ MOST ASKED

A special method with the **same name as the class** and **no return type** that runs automatically when an object is created, to initialise it.

- **Default** — supplied by the compiler only if you write no constructor.
- **No-argument** — written by you without parameters.
- **Parameterised** — takes arguments, e.g. `Student(String name, int age)`.
- **Copy constructor** — built into C++; in Java you write one that takes an object of the same class.

Constructor vs method

Point	Constructor	Method
Name	Same as the class	Any name
Return type	None (not even void)	Must have one (or void)
Called	Automatically on `new`	Explicitly
Inherited	No	Yes

20 What is constructor overloading and constructor chaining?

Overloading: several constructors with different parameter lists. **Chaining**: one constructor calls another — `this(...)` for the same class, `super(...)` for the parent. The call must be the first line.

21 What is the `this` keyword?

A reference to the **current object**. Used to separate a field from a parameter with the same name (`this.name = name`), to call another constructor (`this(...)`), or to pass the current object to a method.

22 What does `static` mean?

A static member belongs to the **class, not to any object** — one shared copy. Static methods can be called without an object (`Math.max()`) and cannot use `this` or non-static members directly. Example: a counter of how many objects were created.

23 Shallow copy vs deep copy? ★ MOST ASKED

Shallow copy copies field values; object references are copied as references, so both copies **share** inner objects — changing one affects the other.

Deep copy also creates **new copies of the inner objects**, so the two objects are fully independent.

Example: copying a Student that has an Address — shallow shares the same Address object; deep creates a new Address.

24 Is there a destructor in Java?

No. Java uses **garbage collection** — the JVM frees objects that have no references. C++ has destructors (`~ClassName()`) for cleanup. In Java, release resources with try-with-resources or a `finally` block.

---

7 · Relationships & design

Association, aggregation, composition

Type	Meaning	Example
Association	Two classes are related and use each other	Teacher — Student
Aggregation	HAS-A, the part **can live independently**	Department has Professors
Composition	HAS-A, the part **dies with the whole**	House has Rooms

25 What are coupling and cohesion?

Cohesion: how focused a class is on one job — aim for **high** cohesion. **Coupling**: how much one class depends on another — aim for **low** coupling. High cohesion + low coupling makes code easy to change and test.

---

26 What are SOLID principles? (asked only if you claim design skills)

- **S**ingle responsibility — one class, one reason to change.
- **O**pen/closed — open for extension, closed for modification.
- **L**iskov substitution — a child object should work wherever the parent is expected.
- **I**nterface segregation — many small interfaces beat one large one.
- **D**ependency inversion — depend on abstractions, not concrete classes.

If asked “Where did you use OOP in your project?”

Name your real classes (e.g. `User`, `Ticket`, `Order`), say which fields are private, where a parent/child relationship exists, and one overridden or overloaded method. A concrete answer from your own project scores far better than a textbook definition.

Rapid-fire one-liners

Question	One-line answer
Can we create an object of an interface?	No — only of a class that implements it (or an anonymous class).
Can a class be both abstract and final?	No — abstract needs a subclass, final forbids one.
Can a constructor be final, static or abstract?	No.
Can a constructor be private?	Yes — used in the Singleton pattern and utility classes.
What is method hiding?	A static method redefined in a child class; the reference type decides which runs.
What is an abstract method?	A method declared without a body; the subclass must implement it.
What is a final variable?	A constant — its value can be assigned only once.
What is an anonymous class?	A class without a name, declared and instantiated in one expression.
What is a Singleton?	A class that allows only one object — private constructor + static getInstance().
What is object cloning?	Creating a copy of an object, e.g. clone() in Java (shallow by default).

Common mistakes interviewers catch

• Saying abstraction and encapsulation are the same — give the hide-complexity vs hide-data difference.

• Saying overloading is possible by return type alone — it is not.

• Claiming Java has no multiple inheritance at all — it has it through interfaces.

• Saying static methods can be overridden — that is method hiding.

• Saying a constructor has a void return type — it has no return type.